CONTENTS

- Overview
- Build timeline
- Key decisions
- Bugs hit & fixed
- The QA pass
- Permissions model
- Still open

DEVBOARD — BUILD JOURNAL

What actually happened building this

A team task tracker (teams → projects → tasks → comments/attachments), built day-by-day as a learning project — Express + raw SQL on the backend, React + Context on the frontend, no ORM, no form library, manual validation before reaching for a library. This is the record of the decisions, the real bugs, and the permission holes that turned up along the way.

Overview

The shape of the thing, and how it was built.

DOMAIN	Teams → Projects → Tasks → Comments / Attachments, three-deep nesting throughout
AUTH	JWT (`Authorization: Bearer`), bcrypt password hashing, token in `localStorage`
ROLES	Global: `admin` / `member` / `viewer` . Per-team: `lead` / `member`
BACKEND	Express, raw `pg` queries, hand-rolled migrations, `express-validator` , `multer`
FRONTEND	React + Vite, `react-router-dom` , Context API for auth, native `fetch`

METHOD	39 tasks across 17 days, each with one manual test before moving on

Build timeline

Highlighted days are where a real decision or a real bug shows up below.

Key decisions

Places where more than one reasonable path existed, and which one got picked.

DECISION · DAY 1

Native Postgres `ENUM` types instead of `CHECK` constraints for `user_role` and `task_status`.

Why: self-documenting in `\d tablename`, stored as a compact 4-byte value. Trade-off accepted: adding a new role/status later needs an `ALTER TYPE` migration, not just an app-level constant.

DECISION · DAY 4

Composite primary key `(team_id, user_id)` on `team_members`, no surrogate id.

It's already the natural key — a user can only join a team once — so it doubles as the uniqueness

constraint. A second add-attempt hits the PK directly (`23505`), no separate app-level duplicate check needed.

DECISION · DAY 6

`roleGuard(allowedRoles)` as a middleware factory, not one fixed middleware.

Calling it with different role arrays per route (`['admin']` vs `['admin','member']`) avoids writing a near-duplicate middleware for every role combination. It only ever reads the global `users.role` — it structurally cannot express "member, but only if team lead."

DECISION · DAY 8

Manual validation first, `express-validator` second — deliberately built twice to compare.

The library version collapsed every hand-rolled `if` / `return` pair into one declarative chain, and moved validation out of controllers into routes entirely. Cost: the inline "why this check exists" comments from the manual version don't have an equivalent home in a `.isInt()` chain.

DECISION · DAY 10

Random generated filenames on disk, never the client's original filename.

Two uploads named `report.pdf` would silently overwrite each other, and a crafted filename could try to escape the `uploads/` folder. Trade-off accepted: the original filename is never stored at all — attachments only ever show the generated name.

DECISION · DAY 11

Centralized error handler + `asyncHandler` wrapper, replacing per-controller `try/catch`.

Express 4 never auto-forwards a rejected promise to error middleware — only a synchronous throw. `asyncHandler` wraps every controller so any rejection reaches `errorHandler.js`, which now owns the Postgres-error-code → HTTP-status mapping in one place instead of five.

DECISION · DAY 13 ASKED, NOT ASSUMED

Token stored in `localStorage`, not an httpOnly cookie — put to the user directly as a real fork.

`localStorage` needs zero backend changes (matches the existing `Authorization: Bearer` design) at the cost of being readable by any script if this app ever had an XSS bug. The cookie alternative would have meant reworking `authMiddleware`, adding `Set-Cookie`, tightening CORS to `credentials: true`, and adding CSRF protection.

DECISION · DAY 16

Raw `XMLHttpRequest` for file uploads, not `fetch`.

`fetch` has no equivalent of `xhr.upload.onprogress` — there's no way to track bytes sent as a request body streams out. `XMLHttpRequest` isn't axios (an even more fundamental native API), so it doesn't break the "no axios" constraint, and the progress bar shows a real byte-count percentage, not a spinner standing in for one.

Bugs hit & fixed

Not hypotheticals — these actually broke a request during testing, before shipping.

BUG · DAY 7

`COALESCE($4, 'todo')` against an `ENUM` column threw `column "status" is of type task_status but expression is of type text`.

Postgres can't infer `$4`'s type inside `COALESCE` against an enum column. Fixed with an explicit cast: `COALESCE($4::task_status, 'todo')` — which, as a side effect, is also what turns an invalid status string into a clean `400` instead of a raw DB error.

BUG · DAY 13

Every frontend `fetch` call would have been silently blocked by the browser — **no CORS headers configured** on the backend.

`curl`/Postman testing never caught this, since CORS is enforced by the browser, not the server. Not in the tech stack, so hand-rolled a small `cors` middleware instead of adding an unlisted dependency — consistent with the "manual before library" pattern already set on Day 8.

BUG · DAY 15 CAUGHT BEFORE SHIPPING

A task object stores `assignee_id` (snake_case, from the DB row) but the `PATCH` body needs `assigneeId` (camelCase).

A single generic "patch these fields" handler would have spread the same object onto both the local UI state and the request body — silently wrong on one

side. Split into two explicit handlers (status, assignee) instead of one shared one.

BUG · DAY 14 CAUGHT BEFORE SHIPPING

`POST /teams`'s response has no `role_in_team` — that field only exists on the `GET /teams` join response.

An optimistic UI update that prepended the raw create-response would have rendered `(undefined)` for a freshly created team's role until the next refresh. Fixed by setting `role_in_team: 'lead'` explicitly, since the backend always makes the creator a lead.

BUG · DAY 11

`register()`'s duplicate-email check ran **outside** the old `try/catch` block — a real unhandled-promise-rejection risk if that query ever threw.

Fixed generically, not by patching that one spot: wrapping the whole function in `asyncHandler` means no future code path in any controller can reintroduce this same class of bug.

The QA pass

Day 17 ran the full admin/member/viewer permission matrix against a live backend — a clean fixture team with all three roles as members, so role was the only variable. 18 checks matched the spec exactly; three did not.

KNOWN GAP · DEFERRED BY DESIGN

A **non-lead member** can still create a project — `roleGuard` reads the global `users.role`, and structurally cannot see `team_members.role_in_team`.

FOUND IN QA · FIXED DAY 17

A member could **delete or edit any task** in a project, not just their own — "Delete task: Member (own)" wasn't enforced.

Fixed with an ownership check against `task.created_by` in `taskController.js`'s `remove()` — admin still bypasses it. Deliberately checked against the creator only: being the **assignee** does not grant delete rights, confirmed directly by assigning a task to a member who wasn't its creator and watching the delete still get rejected.

FOUND IN QA · FIXED DAY 17

`GET /teams/:id` had **no membership check at all** — any authenticated user could view any team's details just by knowing its id.

Built on Day 4, before Day 5 introduced the membership-guard pattern for projects — and never retrofitted. "View everything in *their* team" means their team, not any team. Fixed with the same `isTeamMember` check every other resource already uses.

FOUND IN QA · FIXED DAY 17

`PATCH` / `DELETE /projects/:id` had **no role restriction** — a **viewer** could edit or delete any project on their team.

Projects have no `created_by` column, so an ownership check like the task fix above wasn't

possible. Mirrored the role gate from project *creation* instead (`admin` + `member`, viewer blocked) — verified directly: the viewer account edited a project's description, then deleted a project outright, before the fix.

Permissions model

What's actually enforced, end to end, as of Day 17.

ACTION	ADMIN	MEMBER	VIEWER
Create team	YES	NO	NO
Create / edit project	YES	YES	NO
Delete project	YES	YES	NO
Create / edit task	YES	YES	NO
Delete task	YES	OWN ONLY	NO
Comment on task	YES	YES	YES
View team's data	YES	YES	YES

Every row above also requires team membership — a role alone never grants access to a team you don't belong to.

Still open

Known, documented, not yet closed.

OPEN

OPEN

Member can create a project even without being that team's `lead`.

Adding a team member has no role restriction at all — not a row in the permissions table, left open on purpose.

OPEN

A rejected file upload (failed membership check) still leaves its bytes on disk — no DB row, orphaned file.

OPEN

Attachments never store the client's original filename — only the generated on-disk name is ever shown.

DevBoard build journal — compiled after Task 17.3. Source: `PROGRESS.md` and `DevBoard_PRD.md`.